

Reporte de Actividades

Investigue el concepto de controlador y como funciona en .net , en si es lo que conecta lo que conecta el backend con el frontend, el controlador suele estar usando un servicio, donde el servicio contiene toda la lógica como las consultas y el controlador solo lo llama y puede que de formato o talvez lógica de validaciones, de hecho esto es algo que ya tengo familiarizado por flutter, usaba una arquitectura tipo clean , llegue a usar controladores, servicios , factories, repositories, modelos etc

Vi que en .net había 4 tipos de controladores, pero dos eran para hacer vistas html y el ultimo era la forma que venia haciendo las apis, directamente en program.cs

Y tambien investigue lo de orm, que es para en vez de hacer consultas sql, es solo usar métodos que se podría decir ocultan lo complejo de las consultas, tal vez haciéndolo mas limpio al momento de programar en este caso en c#, trabajando con objetos, me salieron que estos eran los mas conocidos

- Entity Framework Core: el más usado y el recomendado oficialmente por Microsoft para .NET moderno.
- Dapper: más liviano y rápido, pero escribes más SQL manualmente.
- NHibernate: más antiguo y poderoso, pero menos común para proyectos nuevos.
- LINQ to DB: alternativa ligera con buen soporte para consultas.
- ServiceStack OrmLite: otra opción, menos común.

La ia me recomendó directamente entity framework core, ya que es el que recomienda Microsoft

Tambien vi que para consultas complejas, era mejor usar sql puro, y me hace sentido porque por ejemplo en laravel, muchas veces por tratar de usar el orm de laravel se hacia un código bien grandote y feo, muchas veces no lograba hacer lo que quería hacer incluso con la ia, luego se sentía super lento, probablente ahí hubiera sido mejor optar por sql puro

Primero hice el modelo de la tabla de Production.Product llamado Product, que hará referencia a los nombres que tiene a la base de datos, me gusto más de esta forma, porque antes había visto que se podía poner como manualmente como que la referencia a las tablas ahí mismo en el context de la base de datos y no me gusto tanto, se veía sucio, aca directamente en el modelo siento que se ve mas limpio,

```
Program.cs M | ProductosController.cs M | ProductService.cs U | AdventureWorksContext.cs M | Product.cs U X
test > Backend > Models > Product.cs > {} Backend.Models > Product > ProductId : int
1  using System.ComponentModel.DataAnnotations;
2  using System.ComponentModel.DataAnnotations.Schema;
3
4  namespace Backend.Models;
5
6  [Table("Product", Schema = "Production")]
7  public class Product
8  {
9      [Key]
10     [Column("ProductID")]
11     public int ProductId { get; set; }
12
13     [Column("Name")]
14     [MaxLength(50)]
15     public string Name { get; set; } = "";
16
17     [Column("ProductNumber")]
18     [MaxLength(25)]
19     public string ProductNumber { get; set; } = "";
20
21     [Column("Color")]
22     [MaxLength(15)]
23     public string? Color { get; set; }
24
25     [Column("StandardCost", TypeName = "money")]
26     public decimal StandardCost { get; set; }
27
28     [Column("ListPrice", TypeName = "money")]
29     public decimal ListPrice { get; set; }
30
31     [Column("Size")]
32     [MaxLength(5)]
33     public string? Size { get; set; }
34
35     [Column("Weight", TypeName = "decimal(8, 2)")]
36     public decimal? Weight { get; set; }
```

```
Program.cs M  ProductosController.cs M  ProductService.cs U  AdventureWorksContext.cs M  Product.cs U

test > Backend > Data > AdventureWorksContext.cs > { } Backend.Data > AdventureWorksContext > Products : DbSet<Product>

1  using Backend.Models;
2  using Microsoft.EntityFrameworkCore;
3
4  namespace Backend.Data;
5
6  6 references
7  public class AdventureWorksContext : DbContext
8  {
9      0 references
10     public AdventureWorksContext(DbContextOptions<AdventureWorksContext> options)
11         : base(options)
12     {
13     }
14
15     3 references
16     public DbSet<Product> Products => Set<Product>();
17
18     //
19     // protected override void OnModelCreating(ModelBuilder modelBuilder)
20     // {
21     //     modelBuilder.Entity<Product>(entity =>
22     //     {
23     //         entity.ToTable("Product", "Production");
24     //         entity.HasKey(product => product.ProductId);
25     //         entity.Property(product => product.ProductId)
26     //             .HasColumnName("ProductID");
27     //         entity.Property(product => product.Name)
28     //             .HasMaxLength(50);
29     //         entity.Property(product => product.ProductNumber)
30     //             .HasMaxLength(25);
31     //         entity.Property(product => product.Color)
32     //             .HasMaxLength(15);
33     //         entity.Property(product => product.StandardCost)
34     //             .HasColumnType("money");
35     //         entity.Property(product => product.ListPrice)
36     //             .HasColumnType("money");
37     //     });
38     // }
39
40
```

Luego hice el service y hago uso del context de adventure Works para hacer las consultas con el orm

```
Program.cs M  ProductosController.cs M  ProductService.cs U x  AdventureWorksContext.cs M  Product.cs U
test > Backend > Services > ProductService.cs > ...
5 namespace Backend.Services;
7 public class ProductService
8 {
9     4 references
10     private readonly AdventureWorksContext _context;
11
12     0 references
13     public ProductService(AdventureWorksContext context)
14     {
15         _context = context;
16     }
17
18     //task es para que sea asincrono el metodo
19     1 reference
20     public async Task<List<ProductoDto>> GetAllAsync()
21     {
22         return await _context.Products
23             .AsNoTracking() // este es solo se usa para lectura
24             .OrderBy(product => product.Name)
25             .Take(20)
26             .Select(product => new ProductoDto
27             {
28                 ProductId = product.ProductId,
29                 Name = product.Name,
30                 ProductNumber = product.ProductNumber,
31                 ListPrice = product.ListPrice
32             })
33             .ToListAsync();
34     }
35
36     1 reference
37     public async Task<ProductoDetalleDto?> GetByIdAsync(int id)
38     {
39         return await _context.Products
40             .AsNoTracking()
41             .Where(product => product.ProductId == id)
42             .Select(product => new ProductoDetalleDto
43             {
44                 ProductId = product.ProductId,
45                 Name = product.Name,
46                 ProductNumber = product.ProductNumber,
47                 Color = product.Color,
48                 ListPrice = product.ListPrice,
49                 Weight = product.Weight
50             })
51             .FirstOrDefaultAsync();
52     }
53 }
```

Y luego por fin esta el controlador, que solo hace la llamada al método correspondiente del servicio y lo manda con algún mensaje 200 o 404 junto con el resultado si es que hubo

Program.cs M ProductosController.cs M X ProductService.cs U AdventureWorksContext.cs M Product.cs U

test > Backend > Controllers > ProductosController.cs > { } Backend.Controllers > ProductosController

```
1 using Backend.Services;
2 using Microsoft.AspNetCore.Mvc;
3
4 namespace Backend.Controllers;
5
6 [ApiController]
7 [Route("api/productos")]
8 public class ProductosController : ControllerBase
9 {
10     private readonly ProductService _productService;
11
12     public ProductosController(ProductService productService)
13     {
14         _productService = productService;
15     }
16
17     [HttpGet]
18     public async Task<IActionResult> GetAll()
19     {
20         var productos = await _productService.GetAllAsync();
21         //ok devuelve un 200 con el resultado ej :
22
23         return Ok(productos);
24     }
25
26     [HttpGet("{id:int}")]
27     public async Task<IActionResult> GetById(int id)
28     {
29         var producto = await _productService.GetByIdAsync(id);
30
31         if (producto is null)
32         {
33             //not found devuelve un 404
34             return NotFound();
35         }
36
37         return Ok(producto);
38     }
39 }
```

```

Program.cs M  ProductosController.cs M  ProductService.cs U  AdventureWorksContext.cs M  Product.cs U
test > Backend > Controllers > ProductosController.cs > ( ) Backend.Controllers > ProductosController > GetById(int) : Task<IActionResult>
1  using Backend.Services;
2  using Microsoft.AspNetCore.Mvc;
3
4  namespace Backend.Controllers;
5
6  [ApiController]
7  [Route("api/productos")]
8  public class ProductosController : ControllerBase
9  {
10     private readonly ProductService _productService;
11
12     public ProductosController(ProductService productService)
13     {
14         _productService = productService;
15     }
16
17     [HttpGet]
18     public async Task<IActionResult> GetAll()
19     {
20         var productos = await _productService.GetAllAsync();
21         //ok devuelve un 200 con el resultado ej :
22
23         return Ok(productos);
24     }
25
26     [HttpGet("{id:int}")]
27     public async Task<IActionResult> GetById(int id)
28     {
29         var producto = await _productService.GetByIdAsync(id);
30
31         if (producto is null)
32         {
33             //not found devuelve un 404
34             return NotFound();
35         }
36
37         return Ok(producto);
38     }
39

```

Hasta aquí me quede, me faltó ya como usarlo y usarlo en el frontend, por el momento trate de priorizar los conceptos, me sirvió que algunas cosas se parecen pero con distinta sintaxis a flutter y laravel